Responding to type erasure requirements for trivial relocation

Document #:	P4155R0
Date:	2026-03-26
Project:	Programming Language C++
Audience:	EWG

Reply-to:	Oliver Hunt <oliver@apple.com> Corentin Janot <corentin.jabot@gmail.com>

Contents

§ 1 Introduction

P3937 presents a discussion of the type erasure requirements for any future trivial relocation feature in C++. This paper is a very short response to that paper addressing various issue in the presented requirements, and erroneous technical arguments.

This is a first draft response written at late notice in the 2026 Croydon meeting, so may have editing errors, typos, etc.

§ 2 Revision history

— R0 (Croydon): Initial version

§ 3 Motivation

The purpose of this paper is to respond to errors in P3937.

While P3937 is described as a discussion of the requirements of type erased data structures, it is fundamentally just a re-hash of existing papers that are arguing that trivial relocation should be a bitwise copy. As such it should have addressed the previous arguments that resulted in decision for trivial relocation to not have such behavior.

A future revision of this paper will need to address those discussions, and provide evidence that new information is available that will change the outcome of those discussions.

Given the focus of the authors it is understandable that the discussion begins with an example usecase implemented in a way that requires bitwise copy semantics. The problem occurs when the authors leap from existence of that problem in a specific implementation model of that specific data structure, to the assumption the such a model *must* be supported by trivial relocation. The paper does not however include a discussion on what other implementation options are available to type erased containers, and why changing trivial relocation to reduce the number of types that are trivially relocatable is the best solution.

The specific issues surrounding type erased containers were discussed during the standardisation and design process of trivial relocation, and following those discussions the non-bitwise copying semantics were accepted by the committee.

Given that outcome, the burden is on the authors of this paper to demonstrate why the prior decisions were wrong, which will require addressing the previous discussions, and demonstrating why their proposed change is the best possible solution.

Nonetheless there are a number of claims and assertions in the paper that are sufficiently inaccurate if not actually incorrect, and those problems are severe enough that I feel it is necessary to address them immediately.

§ 4 The use case

The goal of P3937 is to have language support for a specific model of implementation of type erased container types, but it does not provide a supporting argument for why such a model should be supported given the existence of other implementation techniques that appear to be able to support the model specified by the trivial relocation proposal.

There are arguments to justify the existence of type erased containers, but C++ has a powerful and robust type system, that supports a wide array of correctly typed containers, and as a general policy C++ favours maintaining that type safety, especially in an era when type safety is increasingly important.

As a result when given the choice our position should be to favour correctly typed data structures over those that are not, especially when the proposals having significant negative consequences for those strongly typed implementations. This impacts the subsequent claims of the paper.

For brevity this paper only addresses the motivation section three, as that contains all the major claims in the paper, as well as the problematic statements.

§ 4.1 Simplicity

The first argument that is made is that having trivial relocation be a bitwise copy is "simpler" than other definitions. This argument of questionable relevance to this paper, as the relevance of the argument is dependent on the presumptive requirement that trivial relocation is an appropriate mechanism

The introductory sentence states

> keeping assumptions minimal to maximize flexibility. By defining relocation as the transfer of object representation, we decouple it from the semantic concept of "movability".

This is actually a statement in support of the previously specified behavior of trivial relocation. P3936 is assuming knowledge of the binary representation of an object. That is not a minimal assumption: by assuming a specific binary representation for objects it restricts what an implementation might do. This is made even more clear when recognizing that the assumption is incorrect: trivial relocation was defined as being an opaque type aware operation specifically because the assumption of bitwise equivalence does not hold.

The result of this incorrect assumption is to directly counter the state goal of decoupling object representation from the concept of movability: the assumption now ties movability of an object not to its definition within the bounds of the abstract machine, but instead to the bitwise representation of the host environment.

The example in this section claims that the object is safe to relocate via memcpy, and that this is simpler. The statements made in no way support this claim. The claim that it can be copied by memcpy is incorrect - use of memcpy to perform relocation is fundamentally UB.

The final argument in this section makes a number of claims I simply do not understand. I don't understand why it claims a move constructor is required, and I cannot work out what optimizations are only able to be performed by limiting the number of types that can be trivially relocated.

In general I would argue that mandating trivial relocation be a bitwise copy makes the feature more complex to end users, by making the nature of whether or not an object is trivially relocatable harder to understand. Instead of being determined by the user's language level definition of a type, it requires them to be aware of the underlying object representation of that type - beneath even, and not defined by, the abstract machine.

§ 4.2 Performance

This section focuses on the performance of type erased data structures. It compares the performance of memcpy with move + destroy. Given the goal of this paper we'll ignore the UB nature of memcpy here, and focus on the latter part.

This paper is specifically stating that in order to achieve a high degree of performance in type erased containers trivial relocatability must be specified as being a purely bitwise copy. To that end this section includes an array of benchmarks.

The referenced benchmarks are comparing memcpy to move-and-destroy so the comparison provides no meaningful performance information. Yes, move-and-destroy, is more expensive than memcpy but this proposal is not arguing about that comparison, it is arguing about trivial_relocation. The benchmarks make no use of trivial relocation to do this work. It is unreasonable to request a performance regressing change to a feature like trivial relocation in order to support code that chooses not to use the feature correctly.

The referenced examples include a large amount of code overly complex code. The included numbers have cases with a 5x performance cost. That kind of delta should not require anything remotely as complicated in order to be reliably demonstrated.

A basic example of a reasonable kind of benchmark (that can obviously be extended to the kinds of examples the authors wish to demonstrate) for this kind of claim can be found on compiler explorer.

In terms of performance P3937 does not improve the performance of type erased container implementations. It equalises the performance with strongly typed containers by forcing them to use move-and-destroy semantics for object types that would otherwise have been trivially relocatable.

Regressing the performance of the preferred model of strongly typed containers, for a single use case is not a reasonable path forward.

§ 4.3 Security

This is actually the most problematic section of the paper. The earlier sections merely result in worse performance, this section actively undermines a widely deployed and powerful platform security feature. It also makes a number of simply incorrect claims. The issues require per error corrections.

§ 4.3.1 Type Erasure vs. Virtual Functions

I am unclear what this section is trying to say, but to address the main points:

1. Virtual member functions are not hard to secure efficiently. As existence proof hardened member function pointers have been deployed at scale for almost a decade. C function pointers, by default, have significantly weaker security properties, not by choice, but as a product of necessity. Using default authenticated

function pointers opens a library to a wide variety of attacks that the arm64e authentication schema is specifically design to prevent.

2. This section makes no sense? Neither the described issue nor the comment on security impact.

This is followed up by a description of how a custom dynamic dispatch method can use a custom authentication schema to protect its internal data structures. Such protection is possible, requires significant care, and when performing manual authentication compiler support is lost. Without an actual detailed description of how this custom authentication would be implemented it is difficult to assess whether the planned design would be sufficient to meet the default levels of protection.

Requiring developers to "manually implement authentication" that would otherwise be handled correctly and automatically is an unreasonable requirement, simply to support a single and less common use case.

But this is also a diversion. P3937 is not concerned with the impact of trivial relocation on the data structures used to implement the type erased dispatch functions. It's concerned about what can actually be stored in those data structures. By mandating trivial relocation be a bitwise copy this entire section loses relevance: types protected by pointer authentication cease to trivially relocatable entirely, and can never be relocated. As in the performance section, the result of this change is a performance regression as it forces types that would otherwise be trivially relocatable to be use move-and-destroy for relocation.

§ 4.3.2 Safety of Fixup APIs

Point 1 is difficult to address as it is a fundamentally incorrect. Simply put this statement

> mismatching these types is permitted at compile-time but causes Undefined Behavior (UB) at runtime if the fixup logic is incorrect.

Is false from start to end. Mismatching the static and dynamic type is an error. It is not permitted at compile time, it is an error that cannot be detected at compile time, no different from any other memory safety bug.

Nor does it become undefined behaviour if the fixup logic is incorrect. It is undefined behavior, and is incorrect.

P3858 was rejected because incorrect fixup logic could cause undefined behavior. P3858 was rejected because it took a class of exploitable type confusion attack that was prevented by pointer authentication, and then specified behavior than would require that protection be removed, and provide a tool that can be used by an attacker to forge a validly signed object from arbitrary memory.

The second point is also completely incorrect. There is no need for this to be studied. It was already implemented prior to trivial relocation being voted down, and the

implementation of trivial relocation was removed from clang.

The final, unnumbered point in this section claimed this trivial relocation builtin a specific model of fixups was “premature and potentially hazardous”. No supporting evidence is made for this claim.

The previously specified trivial relocation semantics do not “bake in” any model of fixup, and in fact trivial relocation made no requirements of how that data was copied.

It is absolutely incorrect to present trivial relocation’s decision to not specify the exact implementation details of how relocation occurs as being somehow more constricting than mandating the exact bit level semantics.

§ 5 Summary

There are an array of problems with P3937. At a basic level it does not address the prior discussions that led to the designed behavior of trivial relocation, and does not provide new information that was not discussed during the design and earlier standardisation work involved in trivial relocation.

Beyond those initial problems, the paper does not present a discussion of any alternative options. Importantly no consideration is given to implementation options for type erased containers. Rather, the paper presents this single method of implementation as a de facto proof that trivial relocation must be a bitwise copy.

As the authors were focused on a use case that is clearly important to them, they did not include any discussion of how this proposal impacts other use cases.

The core claims included a large number of problems: The paper incorrectly claims that the mandate of bitwise copy as being less restrictive and constraining than the design of trivial relocation. The problem here is that while a bitwise copy is intuitively simple, being simple does not mean less constrained. By mandating bitwise copies is inherently more restrictive, and not only does this proposal constrain implementations, it demonstrably prevents relocation of types that were previously relocatable.

The performance comparisons presented did not reflect the actual performance differences of the options, and instead only compared memcpy to the most pessimistic implementation. Such comparison are not sufficient. There are many implementation strategies for type erased containers, and the fact that a single implementation strategy would take a significant performance hit is simply not relevant. Prior feedback from other implementers indicated that there are options beyond simply discarding the idea of relocation. Discussions of performance also need to include the impact on use cases that are not discarding types.

The safety and security section was extremely problematic, and it’s difficult to summarize beyond simply saying that the content was more wrong. The section disregard implementation experience, presented errors in code as being a problems with the

relocation semantics, and while discussing the use of pointer authentication to protect data did not provide sufficient information about what their intended design was. As their model of implementation would require all developers to do similar work, it would be necessary to demonstrate that placing such a security critical burden on all developers is reasonable.